

Iteración 1 — Arquitectura de software de referencia para el desarrollo de sistemas de videovigilancia inteligentes

Maestría en Ciencias de la Computación — Sistemas Distribuidos Móviles y Ubicuos

Moisés Alejandro Sánchez Vergara

Edgar Tobón Sosa

2026-04-07

La **Iteración 1** en ADD 3.0 tiene como objetivo principal establecer la estructura general del sistema (marco de referencia arquitectónico) resolviendo los *drivers* más críticos del negocio. Esta versión preserva la organización original por pasos y amplía cada uno con actividades, decisiones, criterios testables y entregables.

Paso 1: Revisar las entradas

Propósito: Consolidar requisitos, casos de uso, atributos de calidad y restricciones que guiarán el diseño.

Categoría	Detalles
Propósito del diseño	Se trata de un sistema de nueva creación (<i>greenfield</i>) dentro de un dominio maduro. El propósito es producir un diseño arquitectónico (basado en un modelo <i>Edge-Cloud</i>) lo suficientemente detallado para soportar la construcción del sistema, resolviendo problemas críticos como la saturación de red, los altos costos de nube y la dependencia de internet.

Categoría	Detalles
Requerimientos funcionales primarios	• UC-1 (Visualizar cámaras en tiempo real): Porque soporta directamente el modelo de negocio core y está fuertemente ligado a problemas técnicos complejos de latencia (QA-1). • UC-2 (Detectar anomalías): Porque es el núcleo tecnológico (IA) que resuelve el problema de la fatiga de alarmas (falsos positivos) y cambia el enfoque de reactivo a preventivo. • UC-6 (Visualizar grabaciones): Porque es indispensable para el análisis forense post-incidente y está ligado a la restricción de consumo remoto sin depender del almacenamiento del dispositivo cliente (CON-3).
Escenarios de atributos de calidad	Ver la tabla de escenarios detallada más abajo.
Restricciones	Todas las restricciones presentadas (CON-1, CON-2, CON-3) se incluyen como <i>drivers</i> , ya que imponen decisiones innegociables sobre la tecnología de las aplicaciones cliente (Kotlin) y la naturaleza obligatoria del consumo remoto de datos.
Consideraciones arquitectónicas	Todas las consideraciones arquitectónicas (CRN-1, CRN-2, CRN-3, CRN-4) se incluyen como <i>drivers</i> para la estructuración de la interfaz y comunicaciones, definiendo el uso de React/Vue, Kotlin Compose con MVVM y Firebase Cloud Messaging.

Escenarios de atributos de calidad

Con base en los problemas del negocio, los escenarios se han priorizado de la siguiente manera:

ID del Escenario	Atributo de calidad (resumen)	Importancia para el Cliente	Dificultad de Implementación
QA-1	Performance — Latencia E2E < 500 ms	Alta	Alta
QA-2	Modificabilidad — facilidad para cambiar dispositivos/firmware	Alta	Media

ID del Escenario	Atributo de calidad (resumen)	Importancia para el Cliente	Dificultad de Implementación
QA-3	Extensibilidad — agregar sensores/funciona- lidades sin reescribir código	Media	Media
QA-4	Seguridad — Registro y autenticación de dispositivos físicos	Alta	Media
QA-5	Disponibilidad — Funcionamiento local durante pérdida de red	Alta	Alta
QA-6	Ligereza — Visualización y reproducción en dispositivos móviles	Baja	Baja
QA-7	Almacenamiento — Retención y gestión de eventos importantes	Alta	Alta
QA-8	Adaptabilidad — Operar con cambios en el ambiente físico	Alta	Media
QA-9	Performance — Minimizar tráfico de red y costos de nube	Alta	Alta
QA-10	Seguridad — Control de acceso y autenticación de usuarios	Alta	Media
QA-11	Performance — Procesar información registrada sin afectar rendimiento	Media	Media

ID del Escenario	Atributo de calidad (resumen)	Importancia para el Cliente	Dificultad de Implementación
QA-12	Usabilidad — Interfaz intuitiva y eficiencia operativa para usuarios y técnicos	Media	Baja

Paso 2: Establecer el objetivo de la iteración (selección de impulsores)

Está primera iteración de un sistema greenfield, el propósito es establecer la arquitectura general del sistema, teniendo en mente todos los drivers y todos los atributos de calidad:

- **QA-1:** Performance
- **QA-5:** Disponibilidad
- **QA-9:** Performance
- **QA-10:** Adaptabilidad
- **CON-1:** Desarrollo Kotlin
- **CON-2:** Aplicación multiplataforma
- **CON-3:** Consumo remoto de bases

Paso 3: Elegir los elementos del sistema a refinar

Debido a que estamos trabajando un sistema greenfield en la primer iteración, el elemento a refinar es el sistema completo.

Paso 4: Elegir los conceptos de diseño

Arquitecturas propuestas seleccionadas

Arquitectura / Patrón	Por qué se seleccionó
Edge Computing	Reduce latencia E2E y minimiza tráfico de video a la nube, permite grabación y detección local durante cortes de WAN (aborda directamente QA-1, QA-5 y QA-9), soporte para consumo remoto (CON-3) mediante sincronización selectiva.
API Gateway / BFF	Centraliza adaptación de APIs para clientes, gestión de seguridad y límites de acceso; facilita cumplimiento de QA-10 (seguridad) y simplifica integración con clientes móviles/web (CON-2, CON-3).
MVVM (móvil)	Patrón de referencia para separar UI y lógica de presentación en la app móvil; favorece mantenibilidad, testabilidad y usabilidad (QA-12) y cumple con el requisito de desarrollo móvil (CON-1).
SPA (dashboard)	Ofrece experiencia de usuario fluida e interactiva para el panel de administración, facilita visualización en tiempo real y widgets; se integra bien con API Gateway/BFF para adaptar datos al cliente.
Autenticación Stateless	Uso de tokens y validación sin sesión en servidor para escalar autenticación (QA-10), simplificar gestión de sesión y facilitar integración con BFF/API Gateway; trade-off: requiere diseño de refresh tokens y rotación segura.
Store-and-Forward (Almacenamiento y reenvío)	Permite retención local de eventos y reenvío selectivo hacia Cloud, reduciendo tráfico y garantizando disponibilidad durante cortes WAN; trade-off: añadir persistencia y lógica de reconciliación.

Arquitecturas propuestas no seleccionadas

Arquitectura / Patrón	Por qué no se seleccionó
Hub-and-Spoke	Centraliza datos/control; aumenta riesgo de punto único de fallo y requiere inversión en redundancia antes de ser apropiado.
Microservicios + Service Mesh	Requiere madurez operativa (observabilidad, orquestación); se pospone hasta validar carga y necesidades de despliegue independiente.
Event-Driven (Broker)	Añade consistencia eventual y complejidad en garantías; útil para procesos asíncronos, no prioritario para primer PoC que necesita latencia determinista.

Arquitectura / Patrón	Por qué no se seleccionó
Serverless / FaaS	No apto para cargas de streaming continuo; considerado para funciones auxiliares (transformaciones, notificaciones) posteriormente.
Monolito modular	Útil para prototipos rápidos, pero no resuelve drivers críticos de latencia y disponibilidad a escala.
SSR (Server-Side Rendering) para dashboard	Mejora primer paint/SEO pero añade complejidad infra; no prioritario para panel administrativo interno.
PWA (dashboard)	Añade offline y mejora UX móvil; no es requisito crítico para la iteración inicial del panel.
Micro-frontends	Permite despliegue por equipo, pero añade complejidad de integración que retrasaría entregables tempranos.
Widgets / Shell	Extensible, pero orquestación e integración segura entre widgets es trabajo adicional no crítico ahora.
Thin client (móvil)	Depende fuertemente de Edge/Cloud para procesamiento; incumpliría QA-1 en escenarios de latencia estricta.
Local-first / Offline-first (móvil)	Muy valioso para resiliencia (QA-5) pero complica la sincronización; requiere diseñar estrategias de resolución de conflictos.
Feature modules (móvil)	Buena práctica para organización, no es una alternativa arquitectónica que resuelva drivers por sí misma; se adoptará dentro de la implementación.
Streaming/reactive client (móvil)	Cumple QA-1 pero implica mayor esfuerzo de pruebas y control de recursos; se puede incorporar como mejora del cliente en la misma iteración si deciden priorizarlo.
Shell + plugins (móvil)	Extensible y adaptativo, pero añade complejidad de seguridad y compatibilidad que queda para futuras iteraciones.

Diagramas

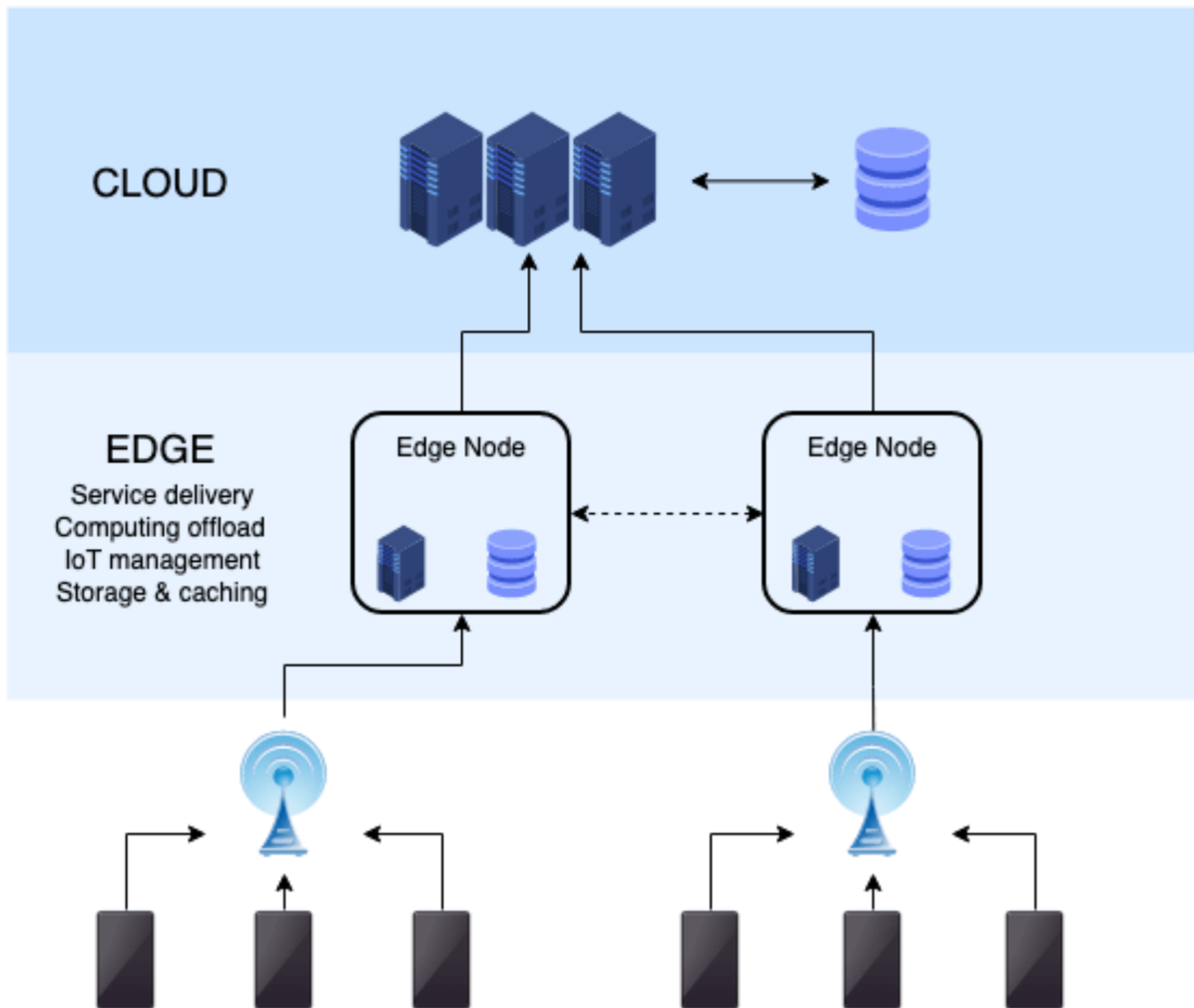


Figura 1: Figura 1. Arquitectura Edge Computing

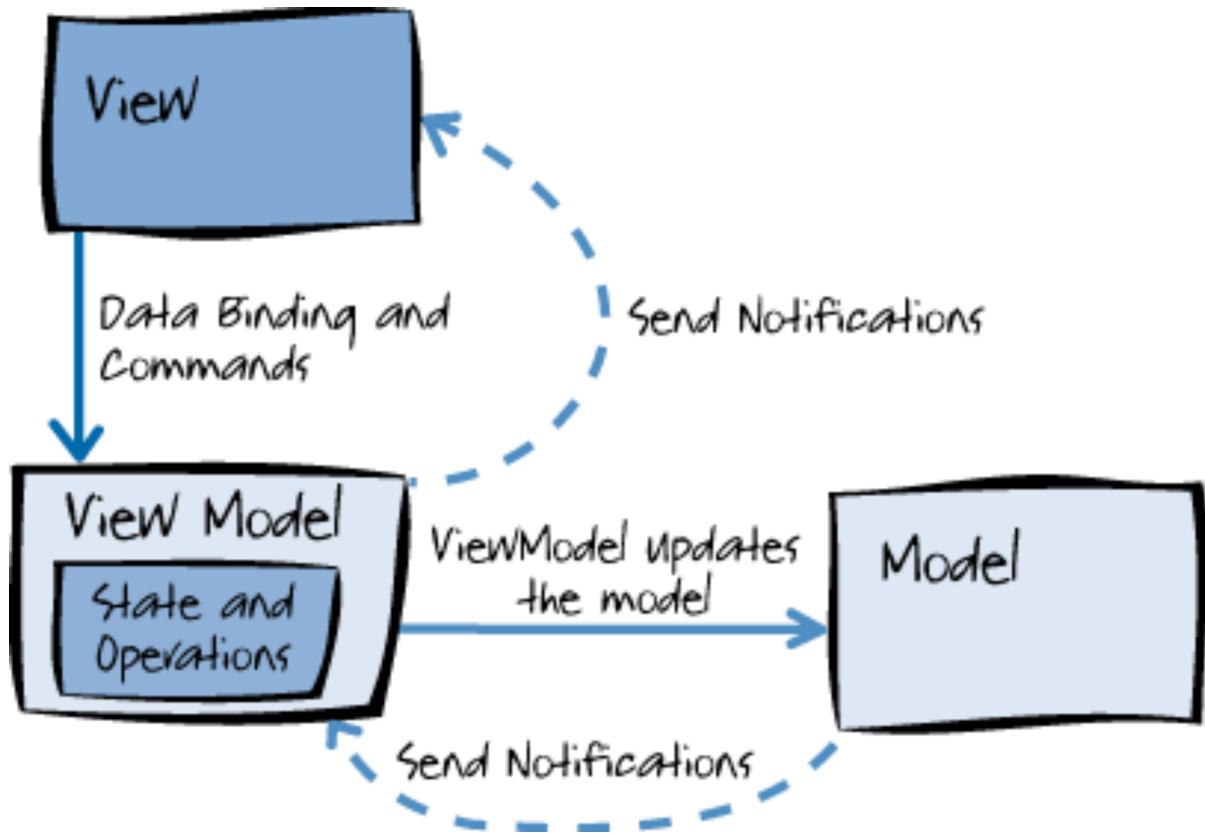


Figura 2: Figura 2. Arquitectura Modelo Vista Vista-Modelo

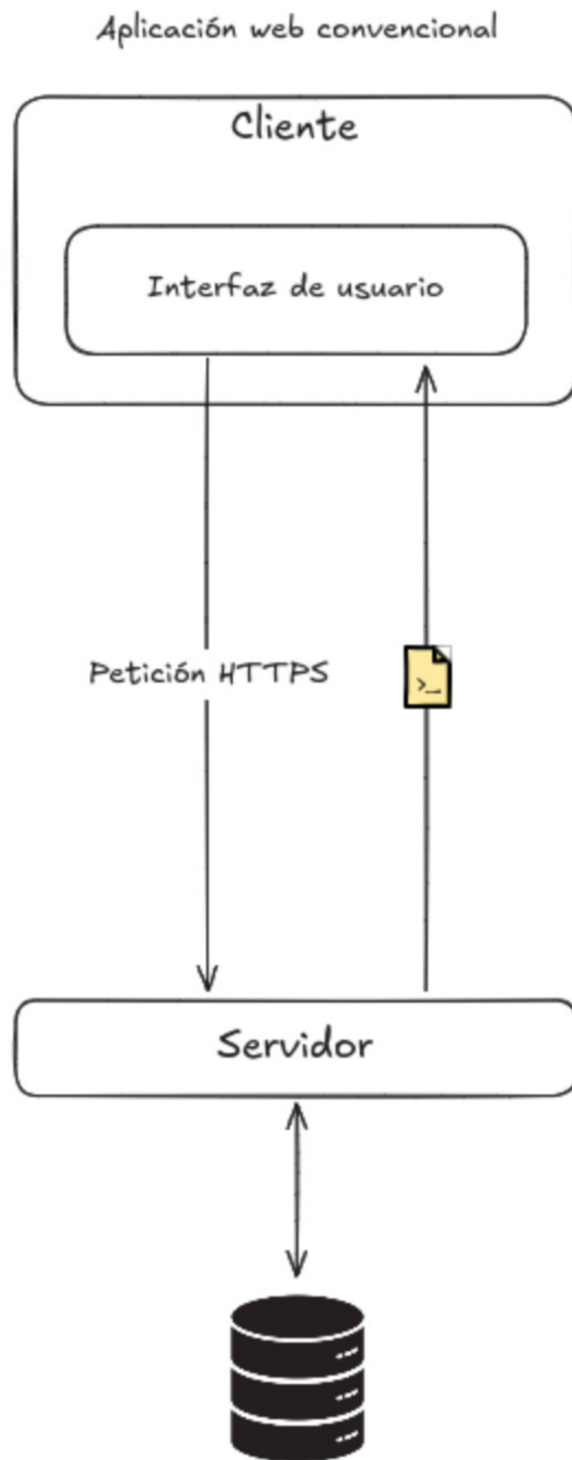


Figura 3: Figura 3. Patrón Single Page Application

Paso 5: Instanciar elementos arquitectónicos y asignar responsabilidades

Módulo / Elemento Instanciado	Responsabilidades Asignadas	Origen (Concepto de Diseño)
Servidor NVR / Nodo Edge	• Ingesta de Video: Capturar y procesar de forma continua el flujo RTSP de las cámaras locales. • Procesamiento AI Local: Ejecutar los modelos de visión computacional para la detección de anomalías en tiempo real sin enviar video a la nube. • Store-and-Forward: Sincronización selectiva en la nube.	Edge Computing
API Gateway Central	• Punto de Entrada Único: Recibir absolutamente todas las peticiones entrantes desde la App Móvil y el Dashboard Web. • Enrutamiento: Redirigir la petición al microservicio interno correspondiente (Dispositivos, Eventos, Notificaciones).	API Gateway
Servicio de Identidad (Auth)	• Validación Stateless: Verificar la firma criptográfica de los tokens en cada petición entrante, confirmando roles y permisos sin necesidad de consultar la base de datos o guardar sesiones en la memoria del servidor.	Autenticación Stateless
Cliente Móvil (Capas View, ViewModel, Model) Dashboard Web (Router, State Store, Componentes)	• Capa Repository: Se encarga de gestionar la lógica de acceso a datos y comunicación con el Nodo Edge. • Router Cliente: Manejar la navegación entre pantallas (Panel de salud, visualización, usuarios) sin recargar la página en el navegador. • State Store: Mantener en memoria RAM del navegador el estado “en vivo” de los Nodos Edge (Online/Offline) para no saturar el servidor con peticiones redundantes. • Componentes UI: Renderizar los <i>videowalls</i> reactivos que se actualizan automáticamente ante cambios en el Store global.	Arquitectura MVVM Patrón SPA (Single Page Application)

Paso 6: Bosquejar vistas y registrar decisiones (ADR)

Edge Computing

Elemento	Descripción / Responsabilidades
Device Layer	Captura los datos con la cámara de videovigilancia y se comunica con la capa Edge.
Edge Layer	Recolecta la información de las cámaras de videovigilancia en tiempo real y se comunica con la capa Cloud
Cloud Layer	Almacena y procesa los datos recibidos desde la capa Edge, proporcionando servicios centralizados y análisis avanzados.
Videocámara y sensores	Se encargan de capturar la información en tiempo real para transmitir la información a la capa Edge.
IA	Se encarga de realizar el procesamiento de la información para el reconocimiento de personas.
Streaming	Expone el endpoint para poder visualizar en tiempo real el video de las cámaras de videovigilancia.
Store and Forward	Se encarga de persistir las grabaciones y sincronizar la información selectiva hacia la capa Cloud.
Almacenamiento	Persiste en la nube la información selectiva y el Dashboard para gestionar el sistema.
Eventos y alertas	Esta capa se encarga de enviar notificaciones hacia la aplicación móvil.
API Gateway	Actúa como punto de entrada único, recibiendo las solicitudes de entrada web y móvil
Autenticación Stateless	Método de autenticación.
Spa (Dashboard)	Aplicación multiplataforma para gestión del sistema.
App móvil	Aplicación móvil para monitoreo del sistema de videovigilancia.

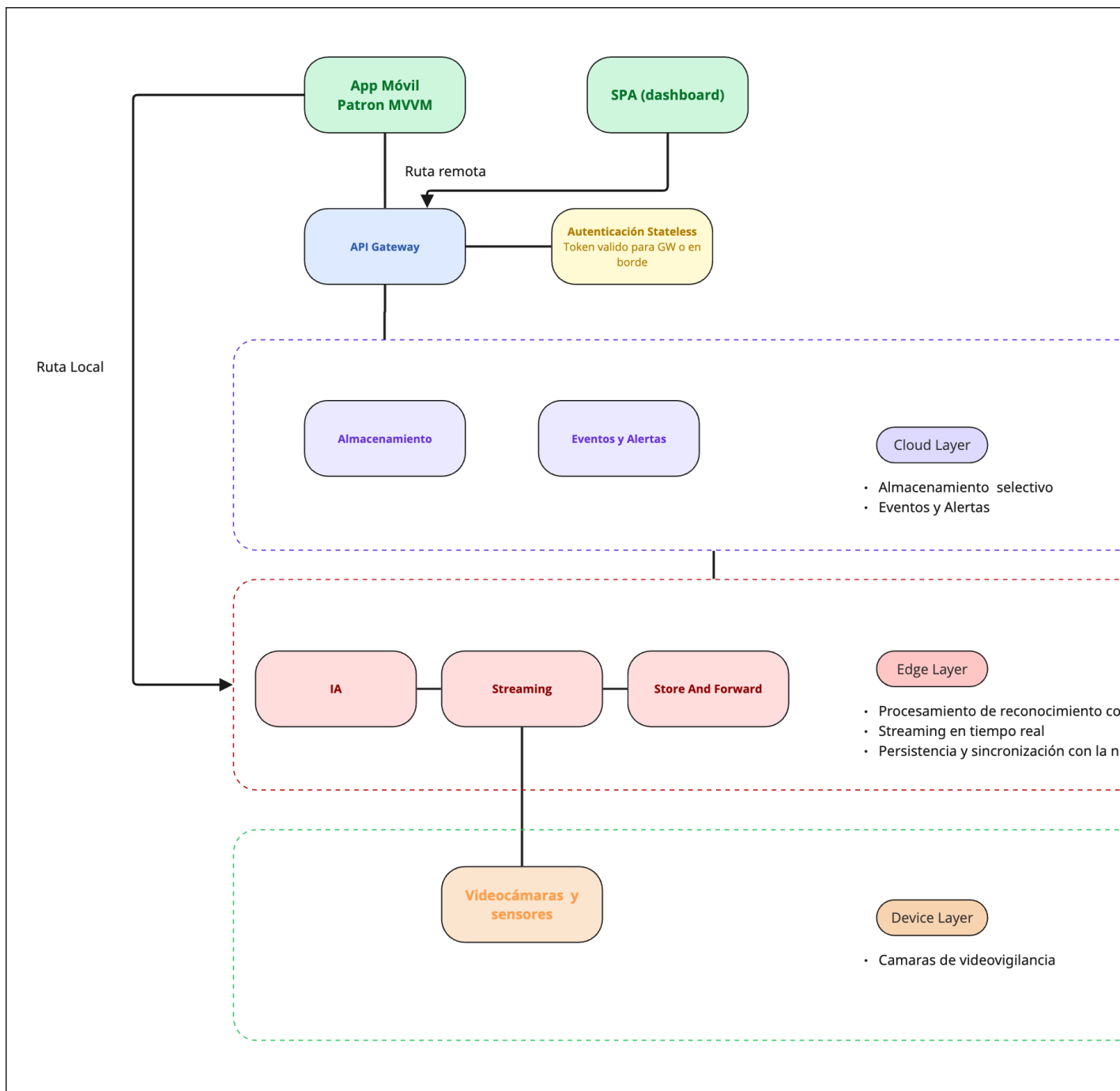


Figura 4: Figura 4. Arquitectura Edge Computing instanciada

MVVM

Elemento	Descripción
View (UI)	Interfaz que renderiza video, controles y alertas.
ViewModel	Orquesta estado reactivo, reconexiones y coordina repositorios.
Repositories / Model	Acceso y persistencia local (DB, cache y cola Store-and-Forward); sincronización con BFF y manejo de datos offline.
API Gateway	Maneja tokens, refresh y permisos; almacenamiento seguro de credenciales y aplicación de políticas de acceso.
Notificaciones	Se recibe notificación cuando el sistema de alertamiento las arroja.

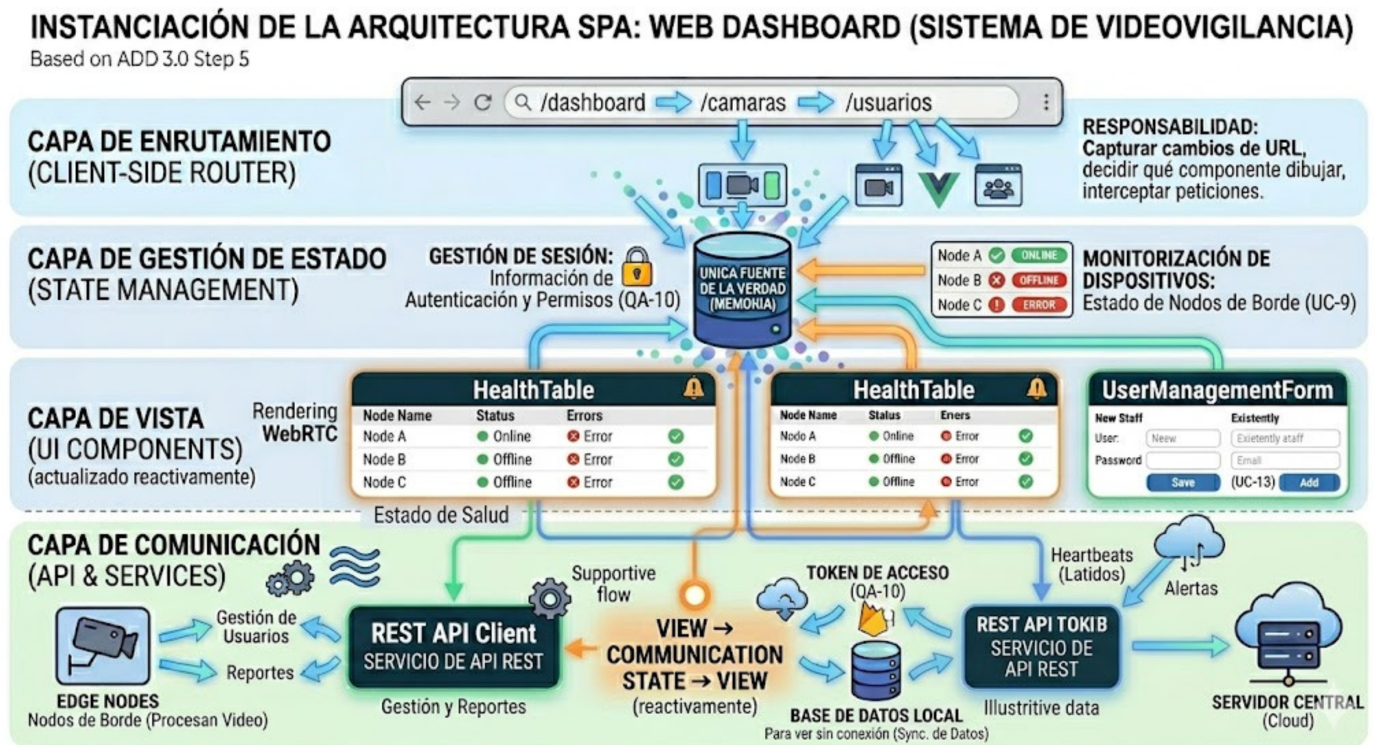


Figura 5: Figura 5. Patrón SPA Instanciado

SPA

Elemento	Descripción
Router (Client-side)	Maneja navegación sin recargas, protección de rutas y carga diferida de módulos.
State Store (Global)	Mantiene el estado en vivo (health, sesiones, videowalls) y sincroniza con BFF mediante sockets/streaming.
UI Components / Videowalls	Componentes reutilizables para renderizar streams y controles, con fallback y optimizaciones de rendimiento.
API Client / BFF Consumer	Orquesta llamadas al BFF, caching y políticas de retry/backoff; alimenta el State Store.
Auth / Session Management	Valida y renueva tokens en cliente, aplica roles y maneja expiraciones de forma segura.

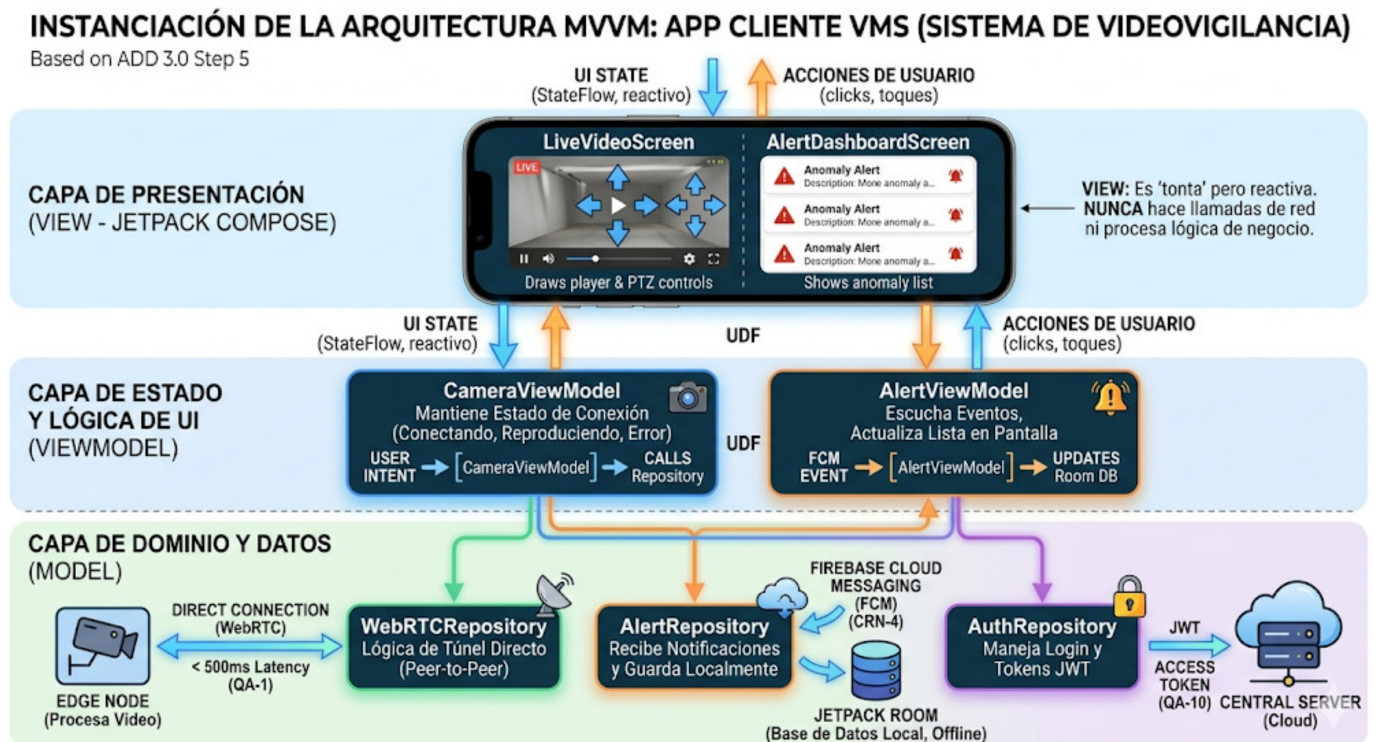


Figura 6: Figura 6. Patrón MVVM Instanciado

Paso 7: Analizar el diseño respecto al objetivo y definir pruebas

Objetivo inicial de la iteración: Establecer la estructura física y lógica general del sistema (Arquitectura Base) para soportar streaming de ultra baja latencia, procesamiento preventivo y operación resiliente ante caídas de red, cumpliendo con las restricciones tecnológicas del equipo.

Resultado de la evaluación: El objetivo se **cumplió satisfactoriamente**. La decisión de adoptar una arquitectura Edge-Cloud estructurada delegó la carga pesada al borde de la red, resolviendo de raíz el problema de saturación y los costos de nube, mientras que la introducción de WebRTC garantizó el cumplimiento de las metas de rendimiento.

Not addressed	Partially addressed	Completely addressed	Design decisions made during the iteration
	UC-1		La arquitectura de referencia seleccionada soporta está funcionalidad
	UC-2		La arquitectura de referencia seleccionada soporta está funcionalidad
	UC-3		La arquitectura de referencia seleccionada soporta está funcionalidad
UC-4			No se tomaron decisiones relevantes, ya que es necesario identificar los elementos que participan en el caso de uso asociado al escenario.
UC-5			No se tomaron decisiones relevantes, ya que es necesario identificar los elementos que participan en el caso de uso asociado al escenario.

Not addressed	Partially addressed	Completely addressed	Design decisions made during the iteration
	UC-6		La arquitectura de referencia seleccionada soporta está funcionalidad.
UC-7			No se tomaron decisiones relevantes, ya que es necesario identificar los elementos que participan en el caso de uso asociado al escenario.
	UC-8		La arquitectura de referencia seleccionada soporta está funcionalidad.
UC-9			No se tomaron decisiones relevantes, ya que es necesario identificar los elementos que participan en el caso de uso asociado al escenario.
	UC-10		La arquitectura de referencia seleccionada soporta está funcionalidad.
	UC-11		La arquitectura de referencia seleccionada soporta está funcionalidad.
UC-12			No se tomaron decisiones relevantes, ya que es necesario identificar los elementos que participan en el caso de uso asociado al escenario.

Not addressed	Partially addressed	Completely addressed	Design decisions made during the iteration
UC-13			No se tomaron decisiones relevantes, ya que es necesario identificar los elementos que participan en el caso de uso asociado al escenario.
UC-14			No se tomaron decisiones relevantes, ya que es necesario identificar los elementos que participan en el caso de uso asociado al escenario.
	UC-15		La arquitectura de referencia seleccionada soporta está funcionalidad.
	QA-1		La arquitectura de referencia seleccionada soporta está funcionalidad.
QA-2			No se tomaron decisiones relevantes, ya que es necesario identificar los elementos que participan en el caso de uso asociado al escenario.
QA-3			No se tomaron decisiones relevantes, ya que es necesario identificar los elementos que participan en el caso de uso asociado al escenario.

Not addressed	Partially addressed	Completely addressed	Design decisions made during the iteration
QA-4			No se tomaron decisiones relevantes, ya que es necesario identificar los elementos que participan en el caso de uso asociado al escenario.
	QA-5		La arquitectura de referencia seleccionada soporta está funcionalidad.
	QA-6		La arquitectura de referencia seleccionada soporta está funcionalidad.
	QA-7		La arquitectura de referencia seleccionada soporta está funcionalidad.
QA-8			No se tomaron decisiones relevantes, ya que es necesario identificar los elementos que participan en el caso de uso asociado al escenario.
	QA-9		La arquitectura de referencia seleccionada soporta está funcionalidad.
	QA-10		La arquitectura de referencia seleccionada soporta está funcionalidad.
	QA-11		La arquitectura de referencia seleccionada soporta está funcionalidad.

Not addressed	Partially addressed	Completely addressed	Design decisions made during the iteration
	CRN-1		La estructura general lógica del sistema ha sido establecida, pero la estructura física aún no ha sido definida.
CRN-2			No se tomaron decisiones relevantes, ya que es necesario identificar los elementos que participan en el caso de uso asociado al escenario.
	CRN-3		La arquitectura de referencia seleccionada soporta está funcionalidad.
CRN-4			No se tomaron decisiones relevantes, ya que es necesario identificar los elementos que participan en el caso de uso asociado al escenario.
CON-1			No se tomaron decisiones relevantes, ya que es necesario identificar los elementos que participan en el caso de uso asociado al escenario.
	CON-2		La arquitectura de referencia seleccionada soporta está funcionalidad.
	CON-3		La arquitectura de referencia seleccionada soporta está funcionalidad.